

AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari



Owl/n

Traceback (most recent call last):
File <stdin>, line 1, in
<module>
StopIteration: invalid system!

Errors



Error Type	Syntax Errors Grammar Failure	Runtime Errors (Exceptions) Meaning Failure	Logic Errors The Silent Killer
The Symptom	The code refuses to even start. It's "un-parsable."	The code starts, then "explodes" mid-execution.	The code runs perfectly, but the result is wrong.
Example	<code>if x = 5:</code>	<code>10 / 0</code>	<code>average = a + b / 2</code>
Detection	Caught by the Python Interpreter instantly.	Caught by the "Traceback" during execution.	Only caught by <i>you</i> (the human) via testing.



1. Syntax Error

```
if True
```

```
    print("Missing colon!")
```

2. Runtime Error

```
def calculate_lexical_density(word_count, total_text):
```

```
    return word_count / total_text
```

```
    # This works... unless total_text is 0.
```

3. Logic Error

```
def get_full_name(first, last):
```

```
    return first + last
```

```
    # Logic Error: We forgot the space between names!
```

```
    # The code runs perfectly, but the output is "JohnSmith" instead of "John Smith"
```



Side Quest!

You write a script to count syllables in a poem. The script runs without crashing, but it claims every single word has exactly 1 syllable. Is this a Runtime Error or a Logic Error? How do you know?

Your reward : 1pt

Errors



Error Type	Description	Common Causes	How to Fix
SyntaxError	Occurs when Python encounters incorrect syntax.	Missing punctuation, extra commas, incorrect function calls, etc.	Check for missing or extra symbols like commas, parentheses, etc.
TypeError	Raised when an operation or function is applied to an object of inappropriate type.	Using incorrect data types for operations, like adding a string to an integer.	Ensure correct data types and convert them if necessary.
IndexError	Occurs when accessing an element in a list, tuple, or string with an invalid index.	Index is out of the valid range of a list or tuple.	Check if the index is within the valid range.
KeyError	Raised when a dictionary key is not found in the dictionary.	Attempting to access a non-existent key in a dictionary.	Ensure the key exists before accessing or use .get() method.
AttributeError	Raised when an invalid attribute reference is made on an object.	Calling a method or accessing an attribute that doesn't exist.	Verify the object has the attribute and check for typos.
FileNotFoundError	Occurs when trying to open a file that doesn't exist.	Trying to open a file that doesn't exist in the specified path.	Ensure the file exists or provide the correct path.
IndentationError	Occurs when there's an unexpected indentation in the code.	Mixing tabs and spaces or incorrect indentation levels.	Use consistent spaces (4 spaces per indentation) and avoid mixing tabs.
NameError	Raised when a local or global name is not found.	Using an undefined variable, misspelling a variable name, or improper imports.	Define the variable or function, check for typos, and import modules properly.
ValueError	Raised when a function receives an argument of the correct type but inappropriate value.	Incorrect values for built-in functions (e.g., passing a negative number to sqrt()).	Ensure the value fits the expected range or constraints.
ZeroDivisionError	Raised when dividing by zero.	Attempting to divide by zero.	Check that the denominator is not zero before performing the division.
OverflowError	Occurs when a calculation exceeds the limits of a data type.	Performing arithmetic that exceeds the range of a number type (e.g., large int).	Use larger data types or split large operations into smaller parts.



```
# 1. IndexError: a list position that is out of bounds.  
phonemes = ["ba", "pa"]  
print(phonemes[5])
```

```
# 2. SyntaxError: The 'Grammar' is fundamentally broken.  
# Missing the closing parenthesis or quote.  
print("Incomplete string"
```

```
# 3. IndentationError: The physical alignment of the logic  
def align_check():  
    x = "LMT"  
    return x #Unexpected indentation here!
```

```
# 4. NameError: Using a variable hasn't been defined.  
result = mystery_variable + 10
```

```
# 5. TypeError: Operation on incompatible types.  
data_point = "Year " + 2026  
# Can't add string and integer.
```

```
# 6. FileNotFoundError: leads to a file that doesn't exist.  
with open("missing_research_data.csv") as f:  
    data = f.read()
```

```
# 7. AttributeError: Asking a feature it doesn't have.  
number = 12  
x= number.append(13)
```

```
# 8. ValueError: Type is right, but content is nonsense.  
age = int("Twenty-Five")  
# int() expects digits, not letters.
```

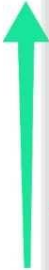
```
# 9. KeyError: Asking a Dict for a key it doesn't hold.  
lexicon = {"noun": "Dog"}  
print(lexicon["verb"])
```

```
# 10. ZeroDivisionError: mathematically impossible.  
frequency = 10 / 0
```

```
# 11. OverflowError: A calculation so large it exceeds  
system limits.  
import math  
result = math.exp(1000)
```

Traceback





```
Runtime Error:
Traceback (most recent call last):
  File "", line 2, in
    print(mylist[10])
IndexError: list index out of range
```

Feature	Technical Name	Clinical Description
The Header	Traceback (most recent call last)	The starting point. It tells you to read from top (oldest) to bottom (newest).
The Stack Trace	Call Stack	A chronological list of every function called leading up to the crash.
The Locator	File "<stdin>", line X	The specific file and line number where the interpreter threw up its hands.
The Diagnosis	Exception Type	The very last line (e.g., IndexError). This tells you <i>what</i> happened.

Traceback



Side Quest!

Traceback (most recent call last):

File `"/path/to/example.py"`, line 4, in `<module>`

`greet('Chad')`

...

File `"/path/to/example.py"`, line 2, in `greet`

`print('Hello, ' + someone)`

read from bottom to top ↑

`name = 'Chad' # name is not defined`

Your reward : 2pt

Traceback



```
def greet(someone):  
    print('Hello, ' + someone)
```

Traceback (most recent call last):

File `"/path/to/example.py"`, line 4, in `<module>`

`greet('Chad')`

...

File `"/path/to/example.py"`, line 2, in `greet`

`print('Hello, ' + someon)`

`NameError: name 'someon' is not defined`

read from bottom to top ↑

Your reward : 2pt

breakpoint()



Command	Action	Technical Description
breakpoint()	Drop a Pin	A built-in function that pauses the program and opens the PDB (Python Debugger).
n (next)	Step Over	Executes the current line and moves to the next one in the same function.
s (step)	Step Into	"Goes inside" a function call to see how it works line-by-line.
c (continue)	Release	Resume normal execution until the next breakpoint or the end of the script.
p <var>	Interrogate	Prints the current value of a specific variable while the code is frozen.

```
def normalize_text(raw_input):  
    # Imagine this is a complex cleaning pipeline  
    step1 = raw_input.strip()  
    step2 = step1.lower()  
  
    breakpoint()  
  
    step3 = step2.replace("!", "")  
    return step3  
  
def linguistic_processor():  
    sample = " Wait! This is a Test! "  
    print("Starting process...")  
    result = normalize_text(sample)  
    print(f"Final Result: {result}")  
  
linguistic_processor()
```

try...except



Keyword	Role	Technical Description
try	The Danger Zone	The block of code where you suspect an error might occur (e.g., reading a file).
except	The First Responder	The block that executes ONLY if a specific error happens inside the try block.
as e	The Evidence	Captures the error message into a variable (usually e) for inspection.
else	The Success Path	Executes only if NO errors occurred in the try block.

try...except



```
def process_corpus_file(filename):
    print(f"--- Attempting to process: {filename} ---")

    try:
        # The 'Danger Zone': This will fail if the file is missing
        with open(filename, 'r') as file:
            content = file.read()
            # Imagine complex linguistic analysis happens here
            print(f"Success! Character count: {len(content)}")

    except FileNotFoundError:
        # The 'Safety Net': Only triggers if the file isn't there
        print(f"CRITICAL ERROR: The file '{filename}' was not found. Skipping...")

    except Exception as e:
        # The 'Catch-All': For any other weird errors we didn't expect
        print(f"ANOMALY: An unexpected error occurred: {e}")
```



Keyword	Role	Technical Description
try	The Action	Attempting a task that might fail (e.g., opening a file).
except	The Crisis	Handling the crash if something goes wrong.
finally	The Guarantee	A block that runs every single time , regardless of success or failure.

```
# The "Janitor" ensures the file closes no matter what
file = open("research_data.txt", "w")
```

```
try:
    # Imagine a crash happens here
    x = 10 / 0
except ZeroDivisionError:
    print("Oops! Math failed.")
finally:
    file.close()
    print("Janitor: File closed safely.")
```

try...except



Side Quest!

Why shouldn't we just wrap our entire program in one giant try...except Exception: pass block to make it "uncrashable"?

Your reward : 1pt

Precision Catching



Technique	Syntax Example	Technical Description
Specific Catch	<code>except ValueError:</code>	Only intercepts ValueError. If a TypeError happens, the program still crashes (as it should!).
Multiple Exceptions	<code>except (EOFError, KeyboardInterrupt):</code>	A "tuple" of errors. Handles several distinct issues with the same backup plan.
Aliasing	<code>except ZeroDivisionError as e:</code>	Saves the specific error message into e so you can log exactly what went wrong.
The Hierarchy	<code>except Exception:</code>	The "Great Umbrella." Catches almost everything. Use this last and with extreme caution.

Precision Catching



```
def calculate_participant_score(data_map, participant_id):  
    try:  
        # Step 1: Accessing the dictionary (Potential KeyError)  
        raw_score = data_map[participant_id]  
        # Step 2: Converting to integer (Potential ValueError)  
        clean_score = int(raw_score)  
        # Step 3: Normalizing (Potential ZeroDivisionError)  
        final_score = 100 / clean_score  
        return final_score
```

```
except KeyError:  
    print(f"ID {participant_id} not found in database. Skipping.")
```

```
except ValueError:  
    print(f"Data for {participant_id} is not a valid number. Check the CSV!")
```

```
except ZeroDivisionError:  
    print(f"Participant {participant_id} has a score of zero. Cannot normalize.")
```

```
except Exception as unexpected:  
    # This is our 'Black Box' recorder for things we didn't see coming  
    print(f"CRITICAL: Something weird happened with {participant_id}: {unexpected}")
```

Validating Input



Technique	Logic Gate	Technical Description
Type Checking	isinstance(data, str)	Verifies the data is the correct "species" before performing operations.
Range Validation	if 0 < x < 120:	Ensures numerical data (like age) exists within physically possible bounds.
Membership Check	if item in valid_set:	Confirms a value belongs to a pre-defined list of allowed categories.
Early Exit	return or continue	Stopping the process immediately if the "contract" of the data is broken.

Validating Input



```
def process_participant_data(name, age, language_code):

    # 1. The Type Check (The 'Species' Gate)
    if not isinstance(name, str):
        print(f"[-] VALIDATION FAIL: Name must be a string, not {type(name)}.")
        return False

    # 2. The Range Check (The 'Physical' Gate)
    if not (0 < age < 110):
        print(f"[-] VALIDATION FAIL: Age {age} is outside research bounds.")
        return False

    # 3. The Membership Check (The 'Category' Gate)
    allowed_langs = ['EN', 'PL', 'RU']
    if language_code.upper() not in allowed_langs:
        print(f"[-] VALIDATION FAIL: {language_code} is not a supported language.")
        return False

    # Logic only executes if all gates are passed
    print(f"[+] VALIDATED: Processing {name} (Age: {age}, Lang: {language_code})")
    return True
```



Keyword	Action	Technical Description
raise	Manual Trigger	Forces an error to occur immediately, stopping execution.
ValueError	The Alarm Type	Specifies the "species" of error to trigger.
("Message")	The Reason	A custom string explaining exactly <i>why</i> the code was stopped.

```
def check_reaction_time(ms):  
    # Python thinks negative numbers are fine. We know better.  
    if ms < 0:  
        raise ValueError("Time travel detected! Reaction time cannot be negative.")  
  
    print(f"Valid data point: {ms}ms")
```



Side Quest!

Why not just use `print("Error")` and `return`? Why is `raise` better for serious research?

Your reward : 1pt

assert



Keyword	The "Human" Translation
assert (The Logic Guard)	"This condition must be true, or this program is a lie."
Condition (The Rule)	The scientific fact you are testing (e.g., <code>length > 0</code>).
Message (The Red Flag)	The specific explanation shown when the rule is broken.

```
def get_average_reaction_time(times):  
    # Sanity Check: We cannot average an empty list!  
    assert len(times) > 0, "Scientific Error: Participant has 0 trials."  
  
    return sum(times) / len(times)
```

Conclusion



Tool	The Strategic Intent	Python Syntax	The "Owl" Rationale
Validation	Gatekeeping: Stop "garbage" data at the entrance.	<code>if x < 0: return</code>	Prevents the program from even starting a doomed task.
<code>raise</code>	Whistleblowing: Signal a scientific violation.	<code>raise ValueError("Msg")</code>	Forces an immediate stop when data "lies" to the science.
<code>try...except</code>	Insurance: Create a safety net for outside risks.	<code>try: ... except Error:</code>	Handles unpredictable events (missing files/web issues).
<code>finally</code>	Cleanup: Ensure the laboratory is left clean.	<code>finally: file.close()</code>	Guarantees resource release, even after a total crash.
<code>assert</code>	Sanity Check: Verify your own internal logic.	<code>assert x > 0, "Error"</code>	Catches bugs in your own design during development.
<code>breakpoint()</code>	Interrogation: Freeze a "living" anomaly.	<code>if rare: breakpoint()</code>	Steps inside the execution to see the "why" of a mystery.

